HW1: README

ML for Finance (Prof. Arratia), BSE 2026

Marco Canal Michael Duarte Gonçalves Olivia Peter

15 May 2026

Project layout

The full report is `solution.pdf` (from `solution.tex`). Code lives in `scripts/`. The committed `data/` folder contains the frozen `goyal-welch2022Monthly.csv` panel. We keep it under version control for reproducibility, so that Exercises 2 and 3 produce the same numbers without an internet round-trip. The `output/` folders (`ex1/`, `ex2/`, `ex3/`) are created on demand by the scripts and hold every CSV, PDF figure and TeX fragment cited in the report.

The four scripts

`exercise_01.py` covers Exercise 1. It pulls daily prices for ten assets from Yahoo Finance over our team epoch (2024-01-01 to 2025-12-31), aligns crypto to the NYSE calendar, computes log returns and RiskMetrics EWMA volatility ($\lambda = 0.94$) at weekly and monthly frequency, then for every ordered pair selects the AIC-optimal VAR lag in $[1,5]$ and runs a Granger F-test at that lag. It produces the four causality tables and the four network plots in `output/ex1/`.

`exercise_02.ipynb` covers Exercise 2. It trains the NN, LSTM and GP grids on the Goyal-Welch panel (`dp`, `svar`, `bm`, plus lags of the target) across the three target specifications, together with the drop-each-indicator sensitivity ablation. It writes to `output/ex2/`.

`exercise_03.py` covers Exercise 3. It fits the plain and Yeo-Johnson ARMA-GARCH baselines on the same chronological split as Exercise 2 and scores them on the same 285-observation test block, so the head-to-head table is apples-to-apples. It writes to `output/ex3/`.

Running the pipeline end-to-end

From a fresh checkout, with Python 3.11 and a TeX Live install:

1. `pip install -r requirements.txt`
2. `python scripts/exercise_01.py` (creates `output/ex1/`)
3. Run `scripts/exercise_02.ipynb` (creates `output/ex2/`, see GPU note below)
4. `python scripts/exercise_03.py` (creates `output/ex3/`)

Measured wall-times. Each script logs its own wall time at the end of the run. The numbers below are from our last full run on one of the authors' laptops: an ASUS ROG Strix G18, Intel i9-14900HX (32 logical cores, 32 GB RAM, Ubuntu 24.04) with an NVIDIA RTX 4080 Laptop GPU (12 GB VRAM) used for the TensorFlow steps. They are indicative only: timings vary with CPU generation, disk speed, Yahoo Finance latency and the GPU on hand.

`exercise_01.py`: 3.7 s.

`exercise_02.ipynb`: 13.6 min end to end on the local RTX 4080. For reference, the local GPU is generally faster than Colab on these small NN and LSTM tensors. Per-cell wall times also print via the `%%time`.

`exercise_03.py`: 7.0 s. (the `auto_arima` stepwise search and the GARCH MLE on the percent-scale residuals dominate the runtime).

Note: running Exercise 2 (the GPU step)

The NN, LSTM and GP grids in `exercise_02.ipynb` finish in a few minutes on a GPU and would take several hours on CPU. The path-setup cell auto-detects Colab vs. a local install.
Option A. Google Colab. Upload `HW1/` to Drive at `MyDrive/ML4F_HW1`, open the notebook from Drive in Colab, *Runtime → Change runtime type* → select a T4 GPU, then *Runtime → Run all*.

Option B. Local NVIDIA GPU. The base `requirements.txt` pins plain `tensorflow`, which is enough for CPU. For GPU acceleration, layer the bundled CUDA wheels on top with `pip install -upgrade tensorflow[and-cuda]`. That pulls in `nvidia-cuda-runtime-cu12`, `nvidia-cublas-cu12`, `nvidia-cudnn-cu12` and friends, so TF self-contains its runtime and no system toolkit is required.

A small caveat on reproducibility: we fix `SEED=7` for NumPy, TensorFlow and PyTorch, but GPU kernels and cuDNN convolutions are not fully deterministic. NRMSE values can move in the third decimal place on a different machine. The report quotes four decimal places only where the ranking is robust to that level of jitter.